Solutions: Day 35 Practice Problems (Hinglish)

Day 35 Git & GitHub challenges ke complete step-by-step commands aur in-depth technical explanations.

Solution 1: Production Hotfix & Branch Isolation Pipeline

1. Branch aur Status Check

```
git branch
git status
```

- **Explanation**: `git branch` dikhata hai ki aap kis branch par ho (`* feature-fraud-v2`), aur `git status` batata hai ki kaun-kaun si files modified hain.

2. Safe Context Switching

- **Risk**: Agar aap adhe code ke sath seedhe `git checkout main` karoge toh Git error dekar abort kar dega ya phir aapka adha code `main` branch mein ghus jayega jisse production kharab ho sakti hai.
- **Kaam Safe Karna**:

  ```
  git add .
  git commit -m "WIP: Fraud detection v2 feature work in progress"
  ```

  (Ya phir `git stash` command chala kar kaam ko temporarily hide kar sakte ho).

3. Hotfix Branch Banakar Bug Fix Karna

```
# Step 1: Main branch mein aao aur latest code lo
git checkout main
git pull origin main

# Step 2: Nayi hotfix branch banao aur enter karo
git checkout -b hotfix-auth-patch

# Step 3: auth.py theek karo aur commit karo
git add auth.py
git commit -m "Patch CVE auth token bypass"
```

4. Merge, Push aur Cleanup

```
# Step 1: Main branch mein wapas aakar merge karo
git checkout main
git merge hotfix-auth-patch

# Step 2: Production fix GitHub par push karo
git push origin main

# Step 3: Kaam khatam hone par hotfix branch delete karo
git branch -d hotfix-auth-patch

# Step 4: Wapas apne feature branch par jao
git checkout feature-fraud-v2
```

Solution 2: Multi-Developer Merge Conflict Resolution

1. Conflict Markers ka Structure

`database.py` ke andar Git yeh symbols lagata hai:

```
<<<<<<< HEAD
DB_PORT = 5433
DB_POOL_SIZE = 50
=======
DB_PORT = 5432
DB_POOL_SIZE = 20
>>>>>>> origin/main
```

- **`<<<<<<< HEAD`**: Aapka local code jo aapne likha tha.
- **`=======`**: Dono codes ke beech ki boundary line.
- **`>>>>>>> origin/main`**: GitHub se aaya hua Engineer B ka code.

2. Manual Resolution

Team ke decision ke according file ko manually edit karein aur saare markers delete kar dein:

```
DB_PORT = 5432
DB_POOL_SIZE = 50
```

3. Finalize Merge Commands

```
# Status check karein
git status

# Resolved file stage karein
git add database.py

# Merge commit karein
git commit -m "Resolve merge conflict in database.py: keep port 5432 and pool 50"

# GitHub par push karein
git push origin main
```

Solution 3: Disaster Recovery & Accidental Secret Commit Mitigation

1. Sabse Pehla Action: API Keys ko Turant Delete/Revoke Karein

Security ka Golden Rule: Agar koi secret password ya API key public GitHub par push ho gayi, toh 1 second ke andar automated bots use chura lete hain.

- Isliye Git mein kuch bhi karne se pehle seedhe OpenAI ya AWS console par jakar wo purani key **Revoke / Invalidate** karein aur nayi key banayein.

2. File ko Computer Par Bacha Kar Git se Hatana

```
git rm --cached .env
```

- `git rm .env` : File ko computer se bhi delete kar deta hai.
- `git rm --cached .env` : File aapke computer par safe rehti hai, sirf Git ke tracking index se delete hoti hai.

3. `.gitignore` Setup

`.gitignore` file ke andar add karein:

```
.env
.env.*
!.env.example
```

(Team ke liye `.env.example` file commit karein jisme dummy values hon).

4. `git revert` vs `git reset` Analysis

- `git revert`: Safe hota hai kyunki history ko delete nahi karta, balki ek naya commit banata hai jo pichli file delete karta hai.
- `git reset --hard`: Puraana commit history se gayab kar deta hai, par iske liye `git push --force` karna padta hai jo team ke baki logon ke repos ko corrupt kar sakta hai.
- **Main Point**: History badlo ya mat badlo, key agar public hui toh use rotate karna 100% zaroori hai!